



Namespaces in Kubernetes

Understanding Kubernetes for Orchestration

Introduction

When working with **large Kubernetes clusters**, managing multiple applications, teams, and resources efficiently can be challenging. This is where **Namespaces** come into play.

Namespaces in Kubernetes allow you to **logically partition a cluster** to organize and manage resources efficiently.

This guide will cover:

- **What are Namespaces?**
 - **Why do we need Namespaces?**
 - **How to create and manage Namespaces**
 - **Practical examples**
 - **Troubleshooting and FAQs**
-

What are Namespaces in Kubernetes?

A **Namespace** is a **virtual cluster within a Kubernetes cluster**. It helps in organizing and managing resources like **Pods, Services, and Deployments**.

Every Kubernetes cluster has at least one **default Namespace**, but we can create multiple Namespaces to **isolate resources** for different teams, projects, or environments.

Why Do We Need Namespaces?

Namespaces help in:



- **Resource Isolation** – Different teams can work in their own Namespace without affecting others.
- **Access Control** – Permissions can be managed at the Namespace level.
- **Better Organization** – Different environments like **dev, test, and production** can be separated.
- **Avoiding Name Conflicts** – Multiple applications can have the same resource names in different Namespaces.

💡 *Think of Namespaces like different folders in a computer, organizing files separately.*

Viewing Existing Namespaces

To see the existing Namespaces in a cluster, use:

```
kubectl get namespaces
```

By default, Kubernetes includes the following Namespaces:

Namespa ce	Purpose
default	The default Namespace where all resources are created unless specified.
kube-syst em	System processes and services run here.
kube-publ ic	Publicly accessible resources (mostly cluster-wide info).
kube-nod e-lease	Helps track node health and status.



Creating a Namespace

Namespaces can be created using **kubectl commands** or **YAML files**.

Method 1: Using **kubectl** Command

To create a new Namespace called **dev-environment**, run:

```
kubectl create namespace dev-environment
```

Verify that the Namespace is created:

```
kubectl get namespaces
```

Method 2: Using YAML Configuration

Create a file named **namespace.yaml**:

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev-environment
```

Apply the configuration using:

```
kubectl apply -f namespace.yaml
```

Verify that the Namespace exists:

```
kubectl get namespaces
```

💡 *Using YAML ensures that the Namespace can be recreated easily when needed.*



Using a Namespace for Deployments

Once a Namespace is created, we can deploy applications inside it.

Deploy an Application in a Specific Namespace

Run the following command to create an **Nginx Deployment in the dev-environment Namespace**:

```
kubectl create deployment nginx-app --image=nginx  
--namespace=dev-environment
```

Verify that the deployment is inside the correct Namespace:

```
kubectl get deployments --namespace=dev-environment
```

Set a Default Namespace for Commands

Instead of specifying **--namespace** every time, we can set a default Namespace using:

```
kubectl config set-context --current --namespace=dev-environment
```

Now, any **kubectl** command will use this Namespace by default.

Switching Between Namespaces

To switch between Namespaces, use:

```
kubectl config set-context --current --namespace=<namespace-name>
```

To check which Namespace you are working in:

```
kubectl config view --minify | grep namespace
```



Deleting a Namespace

To delete a Namespace and all its resources:

```
kubectl delete namespace dev-environment
```



Be careful when deleting a Namespace; all resources inside it will be lost!

Common Issues and Troubleshooting

Problem	Solution
Resources not found in Namespace	Use <code>-n <namespace-name></code> when running <code>kubectl get</code> commands
Cannot delete Namespace	Ensure no finalizers are blocking deletion (<code>kubectl get namespace <namespace> -o yaml</code>)
Pod stuck in Pending	Check available resources using <code>kubectl describe pod <pod-name> -n <namespace></code>

Know More (FAQs)

1. What happens if I don't specify a Namespace?

If no Namespace is specified, the resource is created in the **default** Namespace.

2. Can Namespaces communicate with each other?

Yes, but services need to be accessed using **fully qualified domain names (FQDN)**, like:

```
service-name.namespace-name.svc.cluster.local
```



3. How do I list all resources in a Namespace?

To list all resources in a Namespace:

```
kubectl get all -n <namespace-name>
```

4. Can I set resource limits per Namespace?

Yes, using **ResourceQuota** objects. Example YAML:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: namespace-limits
  namespace: dev-environment
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: "4Gi"
```

Apply it using:

```
kubectl apply -f resourcequota.yaml
```

5. What is the difference between a Namespace and a Cluster?

- **Cluster** – The entire Kubernetes environment.
 - **Namespace** – A logical partition **inside a cluster**.
-



Conclusion

Namespaces in Kubernetes help in **organizing and isolating resources**, making it easier to manage large clusters. They ensure **better resource allocation, security, and multi-tenancy**.

👉 Try creating and managing Namespaces in your Kubernetes cluster to see how they improve resource organization!